

Sorting I - Insertion Sort and Bubble Sort

Shahriar Ivan

Lecturer, Department of CSE, IUT



Sorting

Sorting is the process of:

Taking a list of objects which could be stored in a linear order

$$(a_0, a_1, \dots, a_{n-1})$$

e.g., numbers, and returning an reordering

$$(a'_0, a'_1, \dots, a'_{n-1})$$

such that

$$a'_0 \leq a'_1 \leq \dots \leq a'_{n-1}$$

The conversion of an Abstract List into an Abstract Sorted List

Sorting

In these topics, we will assume that:

- Arrays are to be used for both input and output,
- We will focus on sorting objects and leave the more general case of sorting records based on one or more fields as an implementation detail

Sorting

- Sorting algorithms may be performed **in-place**, that is, with the allocation of at most $\Theta(1)$ additional memory (e.g., fixed number of local variables)
- Other sorting algorithms require the allocation of second array of equal size
 - Requires $\Theta(n)$ additional memory
- We will prefer in-place sorting algorithms

Run-time

- The run time of the sorting algorithms we will look at fall into one of three categories:
 - $\Theta(n)$
 - $\Theta(n \lg(n))$
 - $O(n^2)$
- We will examine average- and worst-case scenarios for each algorithm
- The run-time may change significantly based on the scenario
- Any sorting algorithm must examine each entry in the array at least once
 - Consequently, all sorting algorithms must be $\Omega(n)$
- We will not be able to achieve $\Theta(n)$ behaviour without additional assumptions

Run-time

- We will review the more traditional $O(n^2)$ sorting algorithms:
 - Insertion sort, Bubble sort
- Some of the faster $\Theta(n \lg(n))$ sorting algorithms:
 - Heap sort, Quicksort, and Merge sort
- And linear-time sorting algorithms
 - Bucket sort and Radix sort
 - Here, we must make assumptions about the data

These are all examples of “optimal sorting algorithms”

- There are also “suboptimal sorting algorithms”
 - We will never use them but we still discuss these for completeness

Sub-optimal Sorting Algorithms

Before we look at other algorithms, we will consider the Bogosort algorithm:

1. Randomly order the objects, and
2. Check if they're sorted, if not, go back to Step 1.

Run time analysis:

- best case: $\Theta(n)$
- average: $\Theta(n \cdot n!)$ $n!$ permutations
- worst: $\Theta(\infty)$ unbounded...

Sub-optimal Sorting Algorithms

There is also the Bozosort algorithm:

1. Check if the entries are sorted,
2. If they are not, randomly swap two entries and go to Step 1.

Run time analysis:

- More difficult than bogosort...

Insertion Sort

Consider the following observations:

- A list with one element is sorted
- In general, if we have a sorted list of k items, we can insert a new item to create a sorted list of size $k + 1$

Insertion Sort

For example, consider this sorted array containing of eight sorted entries

5	7	12	19	21	26	33	40	14	9	18	21	2
----------	----------	-----------	-----------	-----------	-----------	-----------	-----------	-----------	---	----	----	---

Suppose we want to insert 14 into this array leaving the resulting array sorted

Background

1. Starting at the back, if the number is greater than 14, copy it to the right
2. Once an entry less than 14 is found, insert 14 into the resulting vacancy



Insertion sort algorithm

For any unsorted list:

Treat the first element as a sorted list of size 1

Then, given a sorted list of size $k - 1$

Insert the k^{th} item in the unsorted list into it into the sorted list

The sorted list is now of size $k + 1$

Insertion sort algorithm

Code for this would be:

```
for ( int j = k; j > 0; --j ) {  
    if ( array[j - 1] > array[j] ) {  
        std::swap( array[j - 1], array[j] );  
    } else {  
        // As soon as we don't need to swap, the (k + 1)st  
        // is in the correct location  
        break;  
    }  
}
```

Implementation

This would be embedded in a function call such as

```
template <typename Type>
void insertion_sort( Type *const array, int const n ) {
    for ( int k = 1; k < n; ++k ) {
        for ( int j = k; j > 0; --j ) {
            if ( array[j - 1] > array[j] ) {
                std::swap( array[j - 1], array[j] );
            } else {
                // As soon as we don't need to swap, the (k + 1)st
                // is in the correct location
                break;
            }
        }
    }
}
```

Runtime Analysis

Let's do a runtime analysis of this code

```
template <typename Type>
void insertion_sort( Type *const array, int const n ) {
    for ( int k = 1; k < n; ++k ) {
        for ( int j = k; j > 0; --j ) {
            if ( array[j - 1] > array[j] ) {
                std::swap( array[j - 1], array[j] );
            } else {
                // As soon as we don't need to swap, the (k + 1)st
                // is in the correct location
                break;
            }
        }
    }
}
```

Runtime Analysis

The $\Theta(1)$ -initialization of the outer for-loop is executed once

```
template <typename Type>
void insertion_sort( Type *const array, int const n ) {
    for ( int k = 1; k < n; ++k ) {
        for ( int j = k; j > 0; --j ) {
            if ( array[j - 1] > array[j] ) {
                std::swap( array[j - 1], array[j] );
            } else {
                // As soon as we don't need to swap, the (k + 1)st
                // is in the correct location
                break;
            }
        }
    }
}
```


Runtime Analysis

This $\Theta(1)$ - condition will be tested n times at which point it fails

```
template <typename Type>
void insertion_sort( Type *const array, int const n ) {
    for ( int k = 1; k < n; ++k ) {
        for ( int j = k; j > 0; --j ) {
            if ( array[j - 1] > array[j] ) {
                std::swap( array[j - 1], array[j] );
            } else {
                // As soon as we don't need to swap, the (k + 1)st
                // is in the correct location
                break;
            }
        }
    }
}
```

Runtime Analysis

Thus, the inner for-loop will be executed a total of $n - 1$ times

```
template <typename Type>
void insertion_sort( Type *const array, int const n ) {
    for ( int k = 1; k < n; ++k ) {
        for ( int j = k; j > 0; --j ) {
            if ( array[j - 1] > array[j] ) {
                std::swap( array[j - 1], array[j] );
            } else {
                // As soon as we don't need to swap, the (k + 1)st
                // is in the correct location
                break;
            }
        }
    }
}
```

Runtime Analysis

In the worst case, the inner for-loop is executed a total of k times

```
template <typename Type>
void insertion_sort( Type *const array, int const n ) {
    for ( int k = 1; k < n; ++k ) {
        for ( int j = k; j > 0; --j ) {
            if ( array[j - 1] > array[j] ) {
                std::swap( array[j - 1], array[j] );
            } else {
                // As soon as we don't need to swap, the (k + 1)st
                // is in the correct location
                break;
            }
        }
    }
}
```

Runtime Analysis

The body of the inner for-loop runs in $\Theta(1)$ in either case

```
template <typename Type>
void insertion_sort( Type *const array, int const n ) {
    for ( int k = 1; k < n; ++k ) {
        for ( int j = k; j > 0; --j ) {
            if ( array[j - 1] > array[j] ) {
                std::swap( array[j - 1], array[j] );
            } else {
                // As soon as we don't need to swap, the (k + 1)st
                // is in the correct location
                break;
            }
        }
    }
}
```

Thus, the worst-case run time is

$$\sum_{k=1}^{n-1} k = \frac{n(n-1)}{2} = O(n^2)$$

Consequences of Our Analysis

Unfortunately, it is not very useful in general:

- Sorting a random list of size $2^{23} \approx 8\,000\,000$ would require approximately one day

Doubling the size of the list quadruples the required run time

- An optimized quick sort requires less than 4 s on a list of the above size

Consequences of Our Analysis

The following table summarizes the run-times of insertion sort

Case	Run Time	Comments
Worst	$\Theta(n^2)$	Reverse sorted
Average	$O(d + n)$	Slow if $d = \omega(n)$
Best	$\Theta(n)$	Very few inversions: $d = O(n)$

Bubble Sort

The second sorting algorithm is the $O(n^2)$ bubble sort algorithm

It uses an opposite strategy from insertion sort

- Suppose we have an array of data which is unsorted:
 - Starting at the front, traverse the array, find the largest item, and move (or bubble) it to the top
 - With each subsequent iteration, find the next largest item and bubble it up towards the top of the array

Bubble Sort

As well as looking at good algorithms, it is often useful too look at sub-optimal algorithms

- Bubble sort is a simple algorithm with:
 - a memorable name, and
 - a simple idea
- It is also significantly worse than insertion sort

Implementation

- Starting with the first item, assume that it is the largest
- Compare it with the second item:
 - If the first is larger, swap the two,
 - Otherwise, assume that the second item is the largest
- Continue up the array, either swapping or redefining the largest item

After one pass, the largest item must be the last in the list

- Start at the front again:
- The second pass will bring the second largest element into the second last position
- Repeat $n - 1$ times, after which, all entries will be in place

Implementation

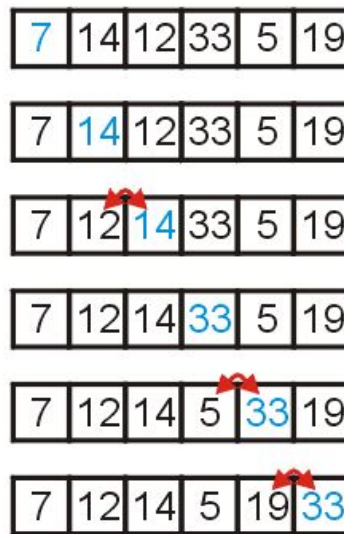
The default algorithm:

```
template <typename Type>
void bubble( Type *const array, int const n ) {
    for ( int i = n - 1; i > 0; --i ) {
        for ( int j = 0; j < i; ++j ) {
            if ( array[j] > array[j + 1] ) {
                std::swap( array[j], array[j + 1] );
            }
        }
    }
}
```

Example

Consider the unsorted array to the right

- We start with the element in the first location, and move forward:
 - if the current and next items are in order, continue with the next item, otherwise
 - swap the two entries



Example

- After one loop, the largest element is in the last location
- Repeat the procedure

7	12	14	5	19	33
---	----	----	---	----	----

7	12	14	5	19	33
---	----	----	---	----	----

7	12	14	5	19	33
---	----	----	---	----	----

7	12	5	14	19	33
---	----	---	----	----	----

7	12	5	14	19	33
---	----	---	----	----	----

Example

- Now the two largest elements are at the end
- Repeat again

7	12	5	14	19	33
---	----	---	----	----	----

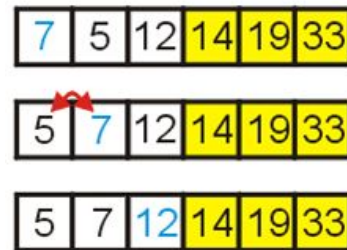
7	12	5	14	19	33
---	----	---	----	----	----

7	5	12	14	19	33
---	---	----	----	----	----

7	5	12	14	19	33
---	---	----	----	----	----

Example

- With this loop, 5 and 7 are swapped



Example

- Finally, we swap the last two entries to order them
- At this point, we have a sorted array

5	7	12	14	19	33
---	---	----	----	----	----

5	7	12	14	19	33
---	---	----	----	----	----

Implementations and Improvements

The next few slides show some implementations of bubble sort together with a few improvements:

- reduce the number of swaps,
- halting if the list is sorted,
- limiting the range on which we must bubble, and
- alternating between bubbling up and sinking down

First Improvement

We could avoid so many swaps...

```
template <typename Type>
void bubble( Type *const array, int const n ) {
    for ( int i = n - 1; i > 0; --i ) {
        Type max = array[0];                // assume a[0] is the max

        for ( int j = 1; j <= i; ++j ) {
            if ( array[j] < max ) {
                array[j - 1] = array[j];    // move
            } else {
                array[j - 1] = max;          // store the old max
                max = array[j];              // get the new max
            }
        }

        array[i] = max;                    // store the max
    }
}
```

Second Improvement - Flagged Bubble Sort

One useful modification would be to check if no swaps occur:

- If no swaps occur, the list is sorted
- In this example, no swaps occurred during the 5th pass

Use a Boolean flag to check if no swaps occurred

3	9	5	1	0	2	6	8	4	7
3	5	1	0	2	6	8	4	7	9
3	1	0	2	5	6	4	7	8	9
1	0	2	3	5	4	6	7	8	9
0	1	2	3	4	5	6	7	8	9

Flagged Bubble Sort

Check if the list is sorted (no swaps)

```
template <typename Type>
void bubble( Type *const array, int const n ) {
    for ( int i = n - 1; i > 0; --i ) {
        Type max = array[0];
        bool sorted = true;

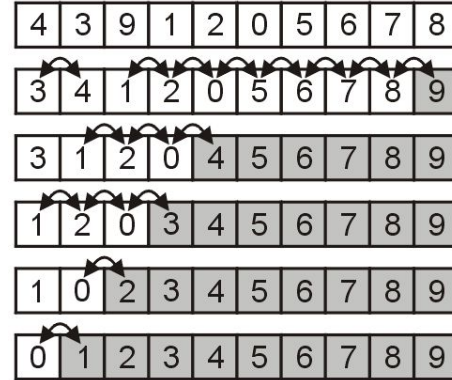
        for ( int j = 1; j < i; ++j ) {
            if ( array[j] < max ) {
                array[j - 1] = array[j];
                sorted = false;
            } else {
                array[j - 1] = max;
                max = array[j];
            }
        }

        array[i] = max;

        if ( sorted ) {
            break;
        }
    }
}
```

Third improvement - Range-limiting Bubble Sort

- Intuitively, one may believe that limiting the loops based on the location of the last swap may significantly speed up the algorithm
 - For example, after the second pass, we are certain all entries after 4 are sorted
- Unfortunately, in practice, this does little to affect the number of comparisons



Range-limiting Bubble Sort

Update i to at the place of the last swap

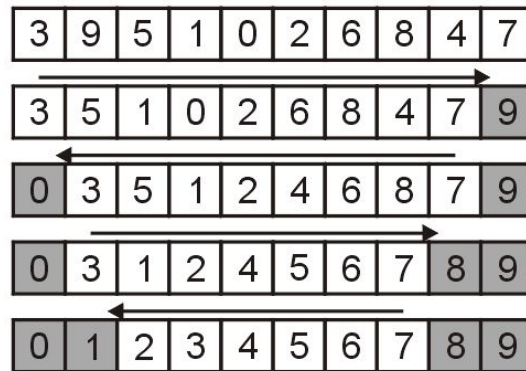
```
template <typename Type>
void bubble( Type *const array, int const n ) {
    for ( int i = n - 1; i > 0; ) {
        Type max = array[0];
        int i = 0;

        for ( int j = 1; j < i; ++j ) {
            if ( array[j] < max ) {
                array[j - 1] = array[j];
                i = j - 1;
            } else {
                array[j - 1] = max;
                max = array[j];
            }
        }

        array[i] = max;
    }
}
```

Alternating Bubble Sort

- One operation which does significantly improve the run time is to alternate between
 - bubbling the largest entry to the top, and
 - sinking the smallest entry to the bottom
- This does, we will see, make a significant improvement



Alternating Bubble Sort

```
template <typename Type>
void bubble( Type *const array, int n ) {
    int lower = 0;
    int upper = n - 1;

    while ( true ) {
        int new_upper = lower;

        for ( int i = lower; i < upper; ++i ) {
            if ( array[i] > array[i + 1] ) {
                Type tmp = array[i];
                array[i] = array[i + 1];
                array[i + 1] = tmp;
                new_upper = i;
            }
        }

        upper = new_upper;

        if ( lower == upper ) {
            break;
        }

        int new_lower = upper;

        for ( int i = upper; i > lower; --i ) {
            if ( array[i - 1] > array[i] ) {
                Type tmp = array[i];
                array[i] = array[i - 1];
                array[i - 1] = tmp;
                new_lower = i;
            }
        }

        lower = new_lower;

        if ( lower == upper ) {
            break;
        }
    }
}
```

Bubble to the back

Sink down to the front

Run-time Analysis

- Because the bubble sort simply swaps adjacent entries, it cannot be any better than insertion sort which does $n + d$ comparisons where d is the number of inversions
- Unfortunately, it isn't that easy:
 - There are numerous unnecessary comparisons

Run-time Analysis

The following table summarizes the run-times of our modified bubble sorting algorithms; however, they are all worse than insertion sort in practice

Case	Run Time	Comments
Worst	$\Theta(n^2)$	$\Theta(n^2)$ inversions
Average	$\Theta(n + d)$	Slow if $d = \omega(n)$
Best	$\Theta(n)$	$d = O(n)$ inversions

End of Lecture